Naive Bayes

Probability-based classifier

CLIFF NOTE

Uses Bayes' theorem to predict class probabilities. Assumes all features are independent — which is rarely true in real life, hence "naive". Despite this, it works surprisingly well for text data.

USE CASE

Email spam detection

Given the words in an email, what's the probability it's spam? Each word is treated independently, e.g. P(spam | 'free', 'win', 'click').

USE THIS WHEN

- √ Text classification — spam filtering, sentiment, topic labelling
- √ Features are roughly independent of each other
- √ You need a fast, interpretable baseline
- √ Dataset is small — NB trains well with limited data

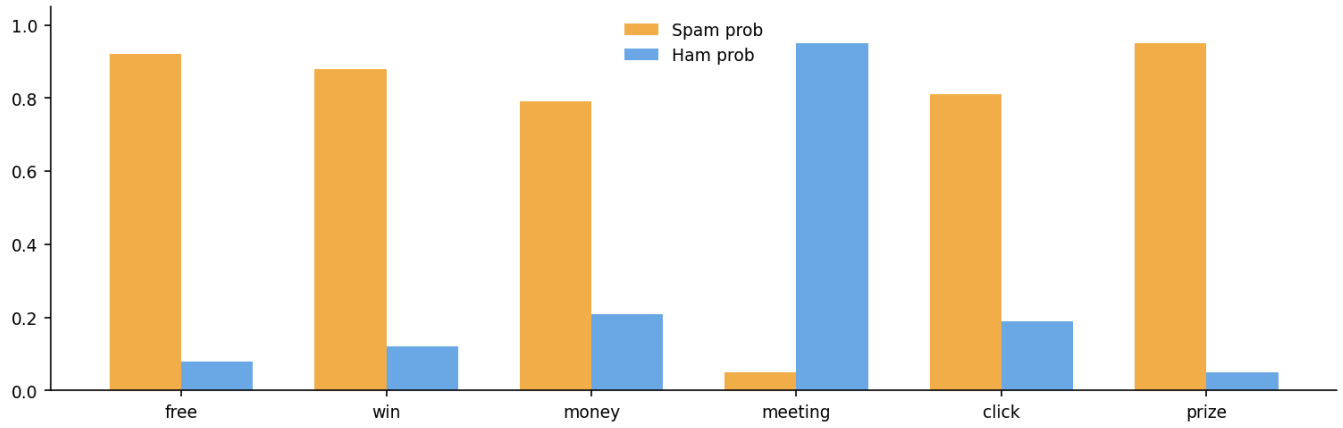
HOW IT WORKS

Counts how often each feature appears per class, then multiplies those probabilities together to score each class. Whichever class scores highest wins. The 'naive' part is assuming features don't influence each other — wrong in practice but works surprisingly well for text data.

CODE

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

emails = ["win free money now", "meeting at 3pm",
          "click here for prize"]
labels = [1, 0, 1]  # 1=spam, 0=ham

vec = CountVectorizer()
X   = vec.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

pred = model.predict(vec.transform(["free prize click"]))
print(pred)  # [1] -> spam
```

VISUAL — WORD PROBABILITIES — SPAM VS HAM



Logistic Regression

Linear classifier

CLIFF NOTE

Fits a linear boundary between classes, then squashes the output through a sigmoid function to get a probability between 0 and 1. Fast, interpretable, and a great baseline for any classification problem.

USE CASE

Customer churn prediction

Given features like usage frequency, support tickets, and contract length — what's the probability a customer will cancel?

USE THIS WHEN

- √ You need probability outputs, not just a class label
- √ The relationship between features and outcome is roughly linear
- √ Interpretability matters — coefficients show feature direction
- √ Always try this as a baseline before reaching for complex models

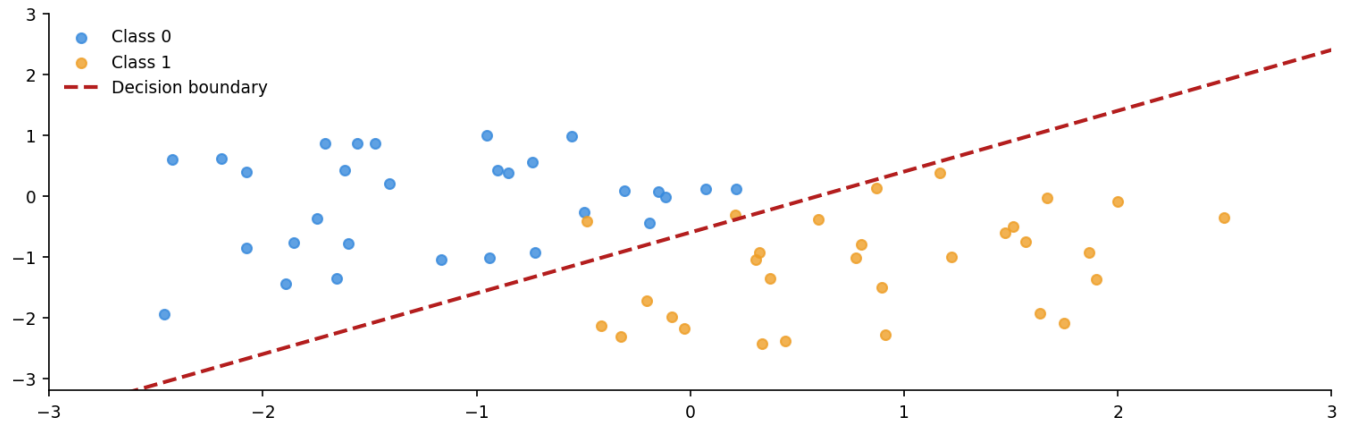
HOW IT WORKS

Computes a weighted score from your features, then squashes it into a 0–1 probability using the sigmoid curve. Training adjusts the weights until the correct class scores near 1. The decision boundary is a straight line (or plane) where the probability is exactly 50%.

CODE

```python
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2,
                           n_redundant=0, random_state=42)
model = LogisticRegression()
model.fit(X, y)

print(model.predict([[0.5, -1.2]]))
# -> [0] or [1]
print(model.predict_proba([[0.5, -1.2]]))
# -> [[0.73, 0.27]]
```

VISUAL — 2D SCATTER WITH DECISION BOUNDARY



Decision Trees

Rule-based tree

CLIFF NOTE

Recursively splits data into branches by asking yes/no questions on features. Very interpretable — you can literally read the rules. Prone to overfitting without pruning.

USE CASE

Loan approval

If income > 50k AND credit score > 700 AND debt ratio < 0.3 -> approve. Each split mirrors how a loan officer actually thinks.

USE THIS WHEN

- √ You need a human-readable model — rules can be printed and shared
- √ Your data has mixed numeric and categorical features
- √ Feature interactions matter more than linear relationships
- √ Quick explainable prototype before trying ensemble methods

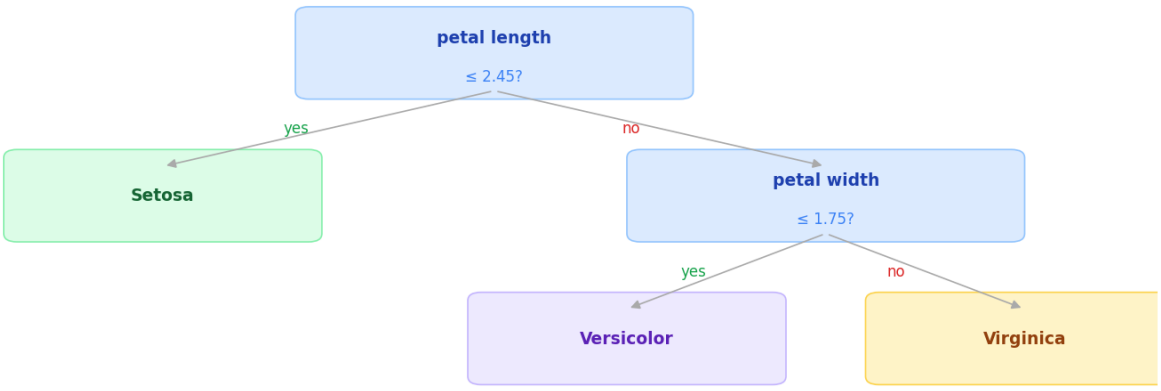
HOW IT WORKS

At each node, tries every possible feature and cut-off value, picking the split that makes the resulting groups most pure (one class dominated). Repeats recursively until groups are pure or a max depth is reached. The result is a set of human-readable if/else rules.

CODE

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris

X, y = load_iris(return_X_y=True)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

rules = export_text(model,
    feature_names=load_iris().feature_names)
print(rules)

print(model.predict([[5.1, 3.5, 1.4, 0.2]]))
# -> [0] = setosa
```

VISUAL — IRIS DECISION TREE STRUCTURE



K-Nearest Neighbors

Instance-based learner

CLIFF NOTE

No training phase — at prediction time it finds the K closest data points and takes a majority vote. Simple and intuitive, but slow on large datasets and sensitive to irrelevant features.

USE CASE

Movie recommendations

Find the K users most similar to you based on viewing history. Recommend what they watched that you haven't.

USE THIS WHEN

- √ Dataset is small to medium — prediction cost grows with data size
- √ Decision boundary is irregular or non-linear
- √ You have no strong assumptions about the data distribution
- √ Avoid with high-dimensional data (curse of dimensionality applies)

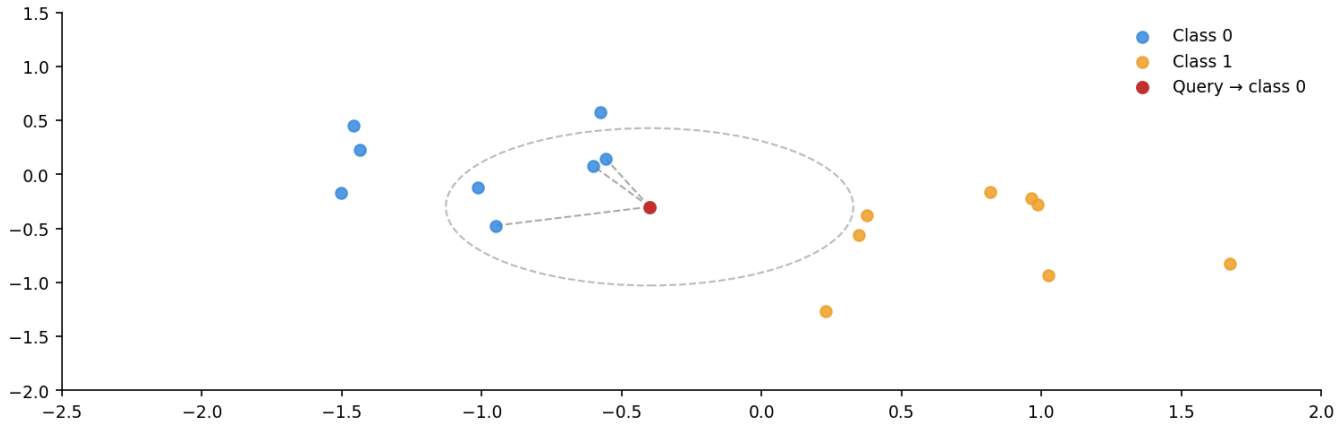
HOW IT WORKS

Stores all training points and does nothing else. At prediction time, measures straight-line distance to every stored point, grabs the K closest, and takes a majority vote. Small K memorises noise (overfit); large K smooths too much (underfit). All the work happens at prediction time, making it slow on large datasets.

CODE

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=300, n_features=2,
                           n_redundant=0, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

print(model.score(X_test, y_test))   # accuracy
print(model.predict([[0.0, 0.5]]))   # class
```

VISUAL — K=3 NEAREST NEIGHBORS — QUERY → CLASS 0



Support Vector Machines

Maximum margin classifier

CLIFF NOTE

Finds the hyperplane that maximises the margin between classes. Uses only the nearest data points (support vectors) to define the boundary. Kernels let it handle non-linear data.

USE CASE

Handwritten digit classification

Works well in high-dimensional spaces with a clear margin. A classic approach for MNIST digit recognition.

USE THIS WHEN

- √ High-dimensional data with fewer samples (e.g. genomics, text with TF-IDF)
- √ Clear margin of separation exists between classes
- √ Non-linear boundaries needed — use RBF or polynomial kernel
- √ Always scale features first — SVM is sensitive to feature magnitude

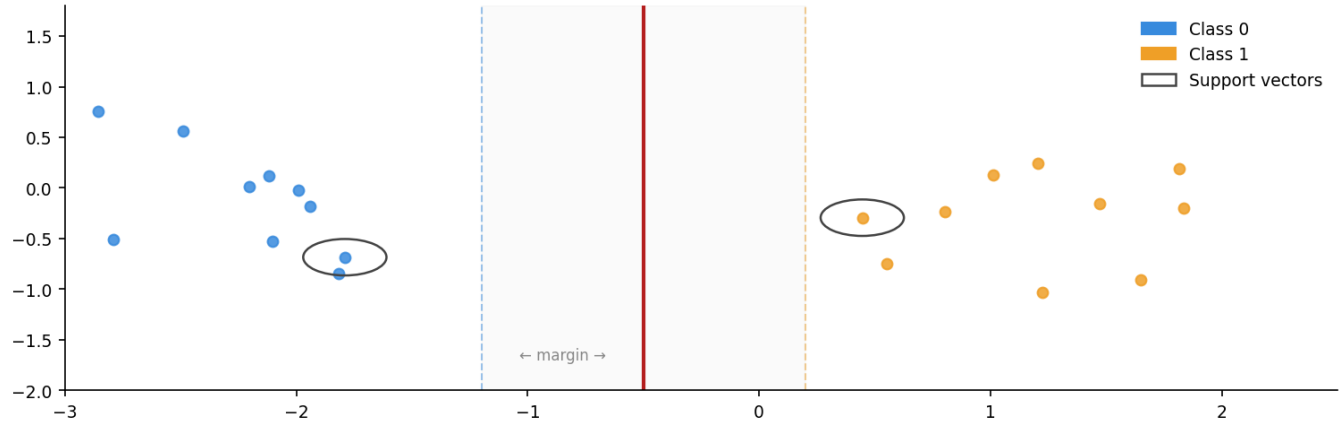
HOW IT WORKS

Finds the widest possible gap (margin) between the two classes. Only the points right on the edge of the gap — the support vectors — matter; the rest of the training data is irrelevant once training is done. Kernels like RBF let it handle cases where no straight line can separate the classes.

CODE

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=300, n_features=2,
                           n_redundant=0, random_state=42)
scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = SVC(kernel='rbf', C=1.0, probability=True)
model.fit(X_scaled, y)

print(model.predict(scaler.transform([[0.5, -1.0]])))
print(f"Support vectors: {model.support_vectors_.shape[0]}")
```

VISUAL — MAXIMUM MARGIN + SUPPORT VECTORS



Neural Networks

Deep learning

CLIFF NOTE

Stacked layers of neurons learn increasingly abstract features. Each layer transforms input via weights + activation functions. The network adjusts weights through backpropagation during training.

USE CASE

Sentiment analysis

Classify customer reviews as positive or negative. The layers learn to detect words, phrases, and context automatically.

USE THIS WHEN

- √ Unstructured data — images, text, audio, time series
- √ Large dataset available — NNs need data to generalise well
- √ Feature engineering is hard — let the network learn representations
- √ Diminishing returns from other models and you can afford training time

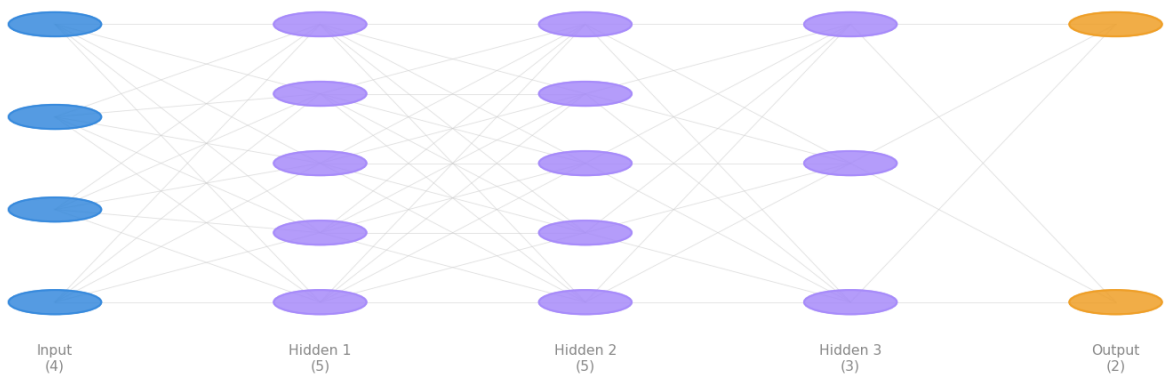
HOW IT WORKS

Passes data through layers of weighted sums followed by non-linear activation functions (e.g. ReLU). Without activations, stacking layers is pointless — they'd collapse to one linear transform. Training uses backpropagation: the chain rule in reverse, nudging weights that contributed to wrong predictions.

CODE

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=500, n_features=4,
                           n_redundant=0, random_state=42)
scaler   = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2 hidden layers: 64 and 32 neurons
model = MLPClassifier(hidden_layer_sizes=(64, 32),
                      activation='relu', max_iter=500,
                      random_state=42)
model.fit(X_scaled, y)

print(model.score(X_scaled, y))   # training accuracy
```

VISUAL — NEURAL NETWORK LAYER ARCHITECTURE



Random Forest

Ensemble — parallel trees

CLIFF NOTE

Builds hundreds of decision trees on random subsets of data and features, then aggregates their votes. The diversity between trees reduces overfitting. One of the most reliable all-round classifiers.

USE CASE

Medical diagnosis

Predict whether a patient has a disease based on many lab values. The ensemble smooths out noise in individual test results.

USE THIS WHEN

- √ Tabular data — this is the default first choice for structured data
- √ You want feature importance with minimal extra work
- √ Overfitting is a problem with a single decision tree
- √ Dataset is large enough for multiple trees but not huge (then use XGBoost)

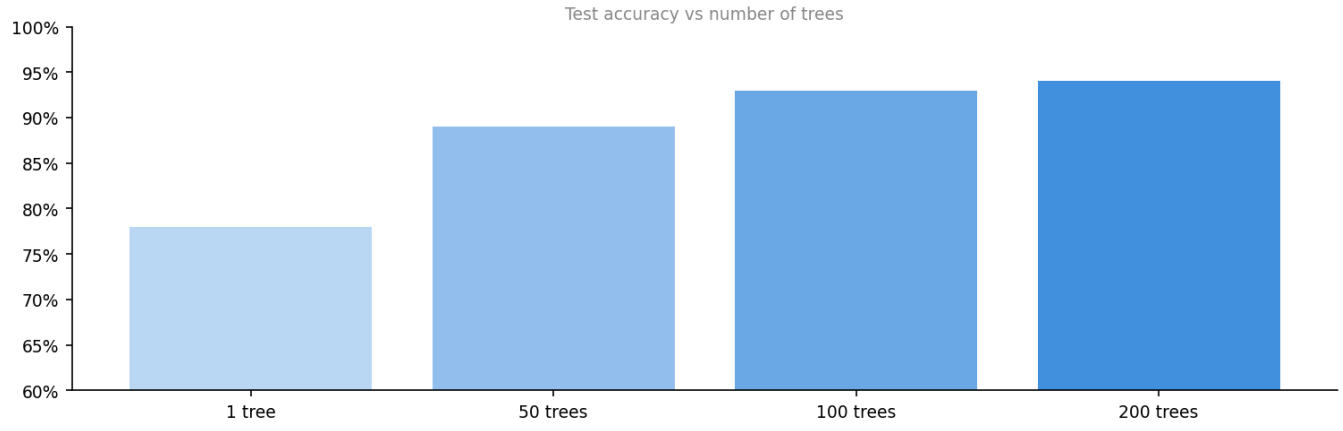
HOW IT WORKS

Trains many decision trees, each on a different random slice of the data and using only a random subset of features per split. Because the trees are diverse, their individual mistakes cancel out when averaged. More trees means more stable predictions, with diminishing returns past ~100–200.

CODE

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=500, n_features=10,
                           n_redundant=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

print(f"Accuracy: {model.score(X_test, y_test):.3f}")
importances = model.feature_importances_
top3 = sorted(enumerate(importances), key=lambda x:-x[1])[:3]
print(top3)  # top 3 most important features
```

VISUAL — ACCURACY VS NUMBER OF TREES



XGBoost / Gradient Boosting

Ensemble — sequential trees

CLIFF NOTE

Builds trees one at a time, where each tree focuses on correcting the errors of the previous. Uses gradient descent to minimise a loss function. Fast, powerful, regularised — Kaggle's favourite.

USE CASE

Credit scoring

Predict whether a loan applicant will default. Handles missing data natively, works well with mixed feature types.

USE THIS WHEN

- √ Tabular data competitions — this is the go-to high-performance model
- √ Dataset has missing values — XGBoost handles them natively
- √ You want fine-grained control over learning rate, depth, regularisation
- √ Random Forest isn't squeezing out enough accuracy

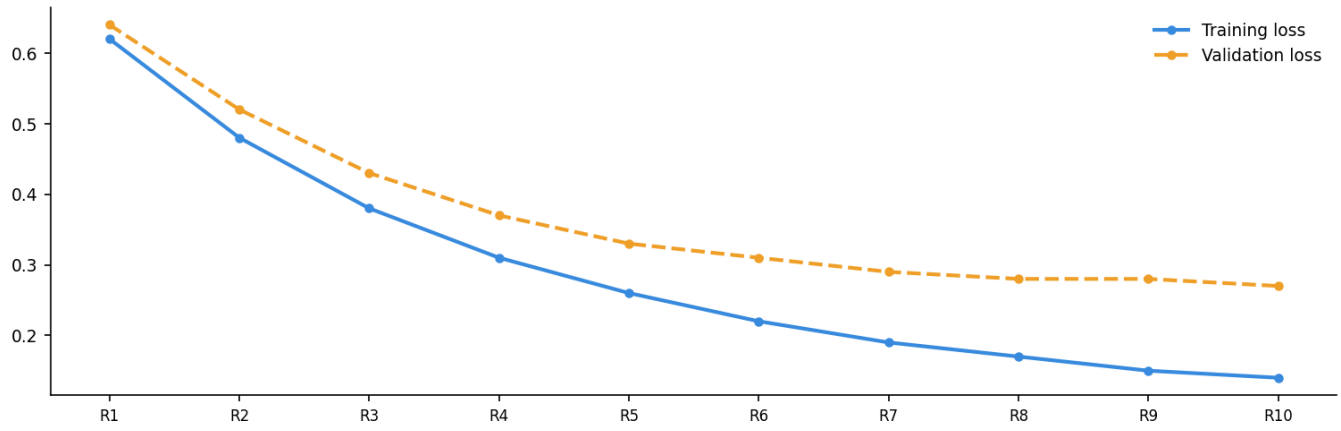
HOW IT WORKS

Builds trees one at a time, each focusing on correcting the mistakes of all previous trees. Uses a small learning rate so each correction is cautious. Regularisation penalises overly complex trees, keeping the ensemble from memorising the training data.

CODE

```
# pip install xgboost
from xgboost import XGBClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=500, n_features=10,
                           n_redundant=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = XGBClassifier(n_estimators=100, learning_rate=0.1,
                      max_depth=4, eval_metric='logloss',
                      random_state=42)
model.fit(X_train, y_train,
          eval_set=[(X_test, y_test)], verbose=False)

print(f"Accuracy: {model.score(X_test, y_test):.3f}")
```

VISUAL — TRAINING VS VALIDATION LOSS PER ROUND



LDA / QDA

Discriminant analysis

CLIFF NOTE

LDA finds a linear combination of features that best separates classes (assumes equal covariance). QDA is the flexible variant allowing each class its own covariance. Both are fast and work well with small datasets.

USE CASE

Face recognition

LDA (Fisherfaces) was a classic approach — project face images onto the dimensions that best separate identities.

USE THIS WHEN

- √ Small dataset — LDA is very data-efficient compared to neural nets
- √ Gaussian class distributions are a reasonable assumption
- √ You also want dimensionality reduction alongside classification
- √ QDA when classes clearly have different spreads or shapes

HOW IT WORKS

Projects the data onto the axis that maximises the separation between class centres relative to how spread out each class is internally. Think of it as finding the viewing angle where overlapping clouds look most distinct. QDA relaxes the assumption of equal spread, giving each class its own shape.

CODE

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
)
from sklearn.datasets import load_wine

X, y = load_wine(return_X_y=True)

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
print(f"LDA accuracy: {lda.score(X, y):.3f}")

qda = QuadraticDiscriminantAnalysis()
qda.fit(X, y)
print(f"QDA accuracy: {qda.score(X, y):.3f}")

X_2d = lda.transform(X)   # collapse to (n_classes-1) dims
print(X_2d.shape)          # (178, 2)
```

VISUAL — 3-CLASS SEPARATION + LD1 AXIS